FH-OÖ Hagenberg/HSD
SDP3, WS 2026
Übung 3

Name: T. Lechner, P. Ackermann — Aufwand in h: 12

Mat.Nr: S2510306017, S2510306001 — Punkte:

Übungsgruppe: 1 — korrigiert:

Beispiel 1 (24 Punkte) Player-Schnittstelle: Sie verwenden in Ihrer Firma HSDSoft einen MusicPlayer von der Firma MonkeySoft. Die öffentliche Schnittstelle des MusicPlayers sieht folgendermaßen aus und kann nicht verändert werden:

```
1     //starts playing with the actual song in list
2     void Start();
3     //stops playing
4     void Stop();
5     //switches to next song and starts at the end with first song
6     void SwitchNext();
7     //gets index of actual song
8     size_t GetActIndex() const;
9     //finds a song by name in playlist
10    bool Find(std::string const& name);
11    //gets count of songs in playlist
12    size_t GetCount() const;
13    //increases the volume relative to the actual volume
14    void IncreaseVol(size_t const vol);
15    //decreases the volume relative to the actual volume
16    void DecreaseVol(size_t const vol);
17    //adds a song to playlist
18    void Add(std::string const& name, size_t const dur);
```

Der MusicPlayer verwaltet Lieder und speichert den Namen und die Dauer jedes Liedes in Sekunden. Er kann gestartet und gestoppt werden und erlaubt das Verändern der Lautstärke. Die Lautstärke ist begrenzt mit 0 und maximal 100. Der Defaultwert für die Lautstärke liegt bei 15.

Zur Simulation liefert der Player je nach Aktion folgende Ausgaben auf der Konsole:

```
playing song number 1: Hells Bells (256 sec)
...
playing song number 4: Hawaguck (129 sec)
...
volume is now -> 70
song: Pulp Fiction not found!
stop song: Hells Bells (256 sec)
...
no song in playlist!
```

In weiterer Folge kaufen Sie einen VideoPlayer der Firma DonkeySoft mit folgender vorgegebenen Schnittstelle:

```
    //starts playing with the actual video in list
    void Play() const;
    //stops playing
    void Stop() const;
    //switches to first video in playlist and returns true, otherwise false if list is empty.
    bool First();
    //switches to next video in playlist and returns true, otherwise false if last song is reached.
    bool Next();
    //returns index of actual video
    size_t ActIndex() const;
    //returns name of actual video
    std::string const ActVideo() const;
    //sets volume (min volume=0 and max volume=50)
    void SetVolume(size_t const vol);
    //gets actual volume
    size_t GetVolume() const;
    //adds a video to playlist
    void Add(std::string const& name, size_t const dur, VideoFormat const& format);
}
```

Der VideoPlayer kann die Formate WMV, AVI und MKV abspielen. Er verwaltet Videos und speichert den Namen und die Dauer in Minuten. Er kann gestartet und gestoppt werden und erlaubt das Verändern der Lautstärke. Die Lautstärke ist begrenzt mit 0 und maximal 50. Der Defaultwert für die Lautstärke liegt bei 8.

Zur Simulation liefert der Player je nach Aktion folgende Ausgaben auf der Konsole:

```
playing video number 1: Die Sendung mit der Maus [duration -> 55 min], AVI-Format
...
playing video number 3: Freitag der 13te [duration -> 95 min], WMV-Format
...
volume is now -> 30
video: Hells Bells not found!
stop video: Pulp Fiction [duration -> 126 min], MKV-Format
...
no video in playlist!
```

Für einen Klienten soll nun nach außen folgende, unabhängige Schnittstelle zur Verfügung gestellt werden:

```
virtual void Play() = 0;
virtual void VolInc() = 0;
virtual void VolDec() = 0;
virtual void Stop() = 0;
virtual void Next() = 0;
virtual void Prev() = 0;
virtual void Select(std::string const& name) = 0;
```

Mit dieser Schnittstelle kann der Klient sowohl den MusicPlayer als auch den VideoPlayer verwenden. Die Methoden `VolInc()` und `VolDec()` erhöhen bzw. erniedrigen die Lautstärke um den Wert 1. `Next()` und `Prev()` schalten vor und zurück. `Select(...)` wählt ein Lied oder ein Video aus der Playliste aus und spielt es ab.

Implementieren Sie alle notwendigen Klassen (auch die Music/VideoPlayer-Klassen) und testen Sie diese entsprechend!

Implementieren Sie eine eigene `PlayerClient`-Klasse, die beide Player testet!

Achten Sie beim Design auf die Einhaltung der Design-Prinzipien und verwenden Sie ein entsprechendes Design-Pattern!

Allgemeine Hinweise: Legen Sie bei der Erstellung Ihrer Übung großen Wert auf eine **saubere Strukturierung** und auf eine **sorgfältige Ausarbeitung!** Dokumentieren Sie alle Schnittstellen und versehen Sie Ihre Algorithmen an entscheidenden Stellen ausführlich mit Kommentaren! Testen Sie ihre Implementierungen ausführlich! Geben Sie den **Testoutput** mit ab!



Systemdokumentation
Projekt Player-Schnittstelle

Version 1.0

T. Lechner, P. Ackermann

Hagenberg, 31. Oktober 2026

Inhaltsverzeichnis

1 Organisatorisches **6**
1.1 Team 6
1.2 Aufteilung der Verantwortlichkeitsbereiche 6
1.3 Aufwand 7

2 Anforderungsdefinition (Systemspezifikation) **7**
2.1 MonkeySoft 7
2.2 DonkeySoft 7
2.3 HSDSoft 8

3 Systementwurf **9**
3.1 Klassendiagramm 9
3.2 Designentscheidungen 9

4 Dokumentation der Komponenten (Klassen) **11**

5 Testprotokollierung **11**

6 Quellcode **11**

1 Organisatorisches

1.1 Team

- T. Lechner, Matr.-Nr.: S2510306017, E-Mail: s2510306017@students.fh-hagenberg.at
- P. Ackermann, Matr.-Nr.: S2510306001, E-Mail: s2510306001@students.fh-hagenberg.at

1.2 Aufteilung der Verantwortlichkeitsbereiche

- T. Lechner
  - Anforderungsdefinition
  - Implementierung und Unit-Test:
    - Video
    - Object
    - IMediaPlayer
    - VideoFormat
    - VideoPlayer
    - VideoPlayerAdapter
- P. Ackermann
  - Designentscheidungen
  - Implementierung und Unit-Test:
    - Song
    - MusicPlayer

* MusicPlayerAdapter
* PlayerClient

1.3 Aufwand

- T. Lechner: geschätzt 6 Ph / tatsächlich 0 Ph
- P. Ackermann: geschätzt 6 Ph / tatsächlich 0 Ph

2 Anforderungsdefinition (Systemspezifikation)

2.1 MonkeySoft

Es soll ein Musik Player implementiert werden. Dieser kann mehrere Songs, bestehend aus Titel und Dauer in Sekunden in seiner Playlist speichern.
Die Songs können abgespielt und das Abspielen wieder gestoppt werden. Es gibt auch die Möglichkeit, auf den nächsten Song zu wechseln, wenn alle Songs durch sind, wird wieder von vorne angefangen.
Der Musik Player verfügt über eine Lautstärke (0 bis 100, standardmäßig 15), die erhöht und verringert werden kann.
Neue Songs können hinzugefügt werden und es kann nach einem bestimmten Titel gesucht werden.
Die Anzahl der Songs in der Playlist und der Index des aktuell ausgewählten Songs kann ermittelt werden.

2.2 DonkeySoft

Es soll ein Video Player implementiert werden. Dieser kann mehrere Videos, bestehend aus Titel, Video-Format (AVI, MKV oder WMV) und Dauer in Minuten in seiner Playlist speichern.

Die Videos können abgespielt und das Abspielen wieder gestoppt werden. Es gibt auch die Möglichkeit, auf das nächste Video zu wechseln und wieder auf den Anfang zurückzuspringen.
Der Musik Player verfügt über eine Lautstärke (0 bis 50, standardmäßig 8), die erhöht und verringert werden kann.
Neue Videos können hinzugefügt werden.
Der Titel und Index des aktuell ausgewählten Videos kann auch ermittelt werden.

2.3 HSDSoft

Es soll für einen Klienten die oben genannten Player in einem einheitlichen Format mit einer gemeinsamen Schnittstelle bereitgestellt werden. Mit dieser Schnittstelle können beide Player gestartet und gestoppt werden. Es kann zwischen Liedern/Videos vor und zurück gewechselt werden. Wird über den Anfang hinaus zurückgewechselt, springt der Player wieder zum Ende. Wird das Ende überschritten, so fängt er wieder von vorne an. Die Lautstärke kann auch immer um 1 erhöht/verringert werden, bis die minimale/maximale Lautstärke erreicht ist. Es kann auch ein ganz bestimmter Titel (sofern dieser existiert) basierend auf dem Namen ausgewählt werden.

Das Hinzufügen neuer Medien ist über die Schnittstelle nicht möglich, sondern nur das Bedienen bereits ausgestatteter Player.

3 Systementwurf

3.1 Klassendiagramm

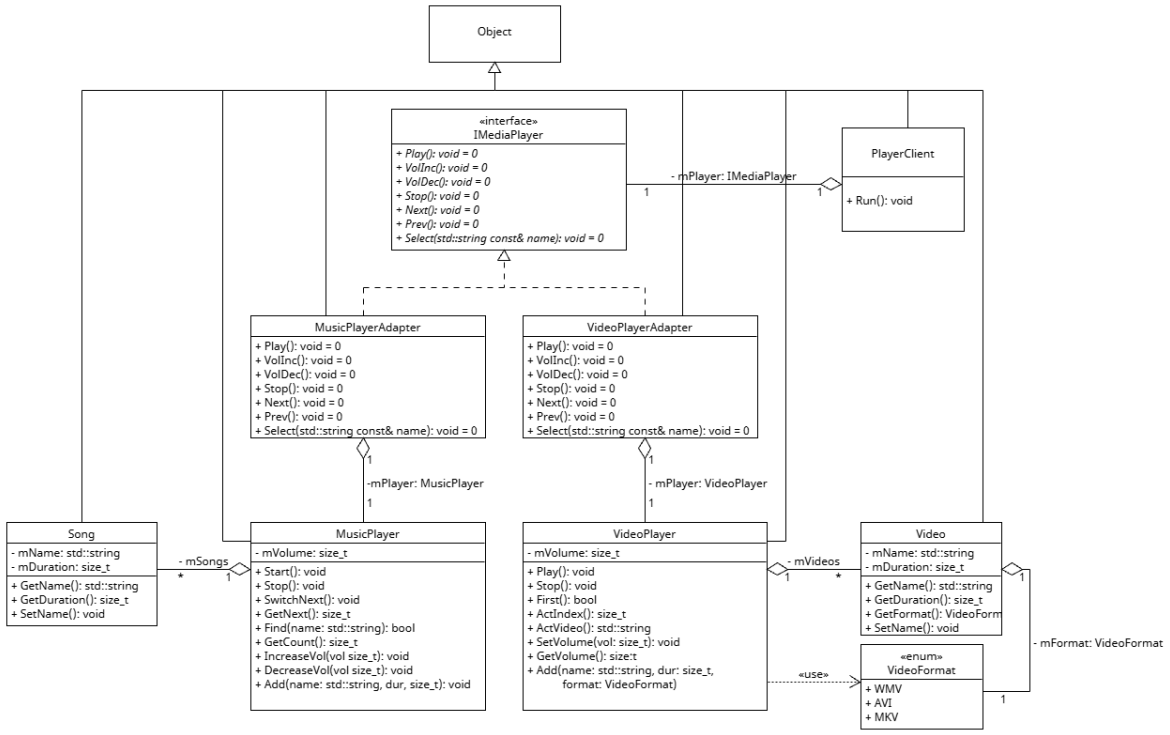


Abbildung 1: Klassendiagramm Player-Schnittstelle

3.2 Designentscheidungen

Um bei weiterer Ableitung der verwendeten Klassen bereits einen virtuellen Destruktor bereitzustellen, werden alle Klassen mit Ausnahme von Interfaces von einer gemeinsamen Object-Klasse mit virtuellem Destruktor abgeleitet.

Da die Klasse Client eine einheitliche Schnittstelle von sowohl dem Music-

Player als auch dem MediaPlayer benötigt, wird hier das Softwarepattern des Adapters angewandt. Hierbei werden für beide Klassen jeweils eine Adapter Klasse bereitgestellt, welche die gemeinsamen Funktionalitäten dem Client bereitstellen sollen. Diese Adapter erben von einem gemeinsamen Interface, damit auch später weitere Adapter für andere Klassen mit ähnlichen Funktionalitäten hinzugefügt werden können. Die Adapter Klassen müssen jeweils Objekte ihrer Klassen, die sie adaptieren aggregieren. So können auf deren Member über ihre Getter zugegriffen werden und auch andere bereitgestellte Methoden verwendet werden.

MusicPlayer:
Um den Zustand der Lautstärke, der über die vorgegebenen Methoden geändert werden soll zu speichern, wird ein Member verwendet. Ebenso wird ein Vector aus „Songs“ zur praktikablen Speicherung der Lieder verwendet. Die Klasse Songs wiederum beinhaltet Member zur Speicherung von Titel und Dauer. Es werden Getter-Methoden zum Lesen zu Verfügung gestellt. Das nachträgliche Ändern soll nur für den Namen möglich sein, da die Länge eines Songs als konstant angenommen wird.

VideoPlayer:
Ebenso wie im MusicPlayer muss auch hier der Zustand der Lautstärke und die einzelnen Videos gespeichert werden können. Für ein Video werden Member für Namen und Länge, aber auch für das Dateiformat benötigt. Die Werte für dieses Dateiformat, werden über ein Enum „Videoformat“ definiert.

Hinweis: Die Adapter für den MusicPlayer und den VideoPlayer werden wie in der Angabe gefordert über die Klasse PlayerClient ausführlich getestet. Da einige Methoden der Klassen, die die Adapter aggregieren von den Adaptermethoden nicht verwendet werden, müssen diese Klassen (MusicPlayer, Song, VideoPlayer, Video) eigenen Unit-Tests unterzogen werden.

4 Dokumentation der Komponenten (Klassen)

Die HTML-Startdatei befindet sich im Verzeichnis ./../doxy/html/index.html

5 Testprotokollierung

6 Quellcode